Database Views are not currently supported natively by Active Record. However, in this section you'll create views to use with Rideshare. Since views have a definition that you may want to evolve over time, you'll manage the lifecycle of views using Rideshare. PostgreSQL does not offer a built-in way to manage versioned views over their lifecycle, so using the client application to do this provides a solution.

For Rideshare you'll use a third party Ruby gem that extends Active Record, adding support for Database Views.

Some of the new terminology in this section is listed below.

Administration Terminology



- Database View — Encapsulates a SQL query
- Materialized View — A View with pre-calculated results to query

A database view encapsulates (Wikipedia: *Encapsulation*[18]) a SQL query and gives it a name. Imagine that you frequently queried New Drivers in Rideshare and wanted to have a shared definition of what a new driver is as a query that could then be re-used for other purposes. You might create a view called "new_drivers" with a query like this one:

```
CREATE VIEW new_drivers AS
    SELECT * FROM users
    WHERE type = 'Driver'
    AND created_at >= (NOW() - INTERVAL '30 days');
```

With that view created in PostgreSQL, you're now able to write a very simple query, `SELECT * FROM new_drivers;`.

Unless views are Materialized, they execute live queries. What does Materialization mean? A Materialized view is a type of View that means the query can be run in advance and then queries to the view – like the one above that queries the pre-calculated results instead of running a live query.

Since the query results are pre-calculated with materialization, querying a materialized view is very fast. The trade-off is that the query results become stale. Fortunately Materialized Views can be refreshed CONCURRENTLY. Let's see how to create both types of views and keep materialized views refreshed.

Database views have a definition and you may want to evolve the definition over time. While there's no built-in solution to PostgreSQL or Ruby on Rails

18. https://en.wikipedia.org/wiki/Encapsulation_(computer_programming)